What AI actually is

Four eras, one ticket routed four ways, and the first call to a model.

60 min | Recorded | Class 1 of 15

1

What this session covers

01
How this track works
5 slides

02
Four eras
6 slides

03
Training and inference
3 slides

04
Code walkthrough
5 slides

© 2026 Devansh Parapalli

Section 1 of 4

How this track works

Fifteen classes, one company, one idea running through all of them.

01
Every class runs the same way

02
We are Lafayette O'Reilly Logistics

03
Who we deal with, and why they show up in our tickets

04
The systems our tickets are about

© 2026 Devansh Parapalli

Every class runs the same way

1
30 min on the idea and the steps, for everyone

2
20 min on the code, for developers and support

3
10 min of questions

4
Exercises live in the repo, for after class

© 2026 Devansh Parapalli

We are Lafayette O'Reilly Logistics

a global freight and logistics company. For the next fifteen classes, this is our company. Yes, the initials are on purpose.

01 What we do
Move freight: bookings, customs, warehouses, trucks. Plus everything behind it: finance, HR, IT, facilities.

02 Why it's hard
Freight across 40+ countries, money in 11 currencies, and a short payment on nearly every cross-border receipt.

03 What's real
Nothing. The company, the systems, the people and the numbers are made up, so we can use them freely.

© 2026 Devansh Parapalli

Who we deal with, and why they show up in our tickets

Bayerische Unfug Maschinen (B.U.M.)	Customer. A German machinery maker that ships its machines to the world through us. Pays in EUR from Munich, usually a little short after bank charges.
Armadillo Security Services (A.S.S.)	Supplier. Physical security and background checks for our warehouses and drivers. Invoices through InvoiceHub, one per check or per guard shift.
Drymark Institutional Handling (D.I.H.)	Supplier. Admin, facilities and janitorial across our offices.
Gastro Alliance Group (G.A.G.)	Supplier. Catering. Owns the coffee machines, which people still raise IT tickets about.
Stratocumulus Cloud (S.C.)	Supplier. Our cloud provider. The bill needs cost centres it never has; credits arrive late.

© 2026 Devansh Parapalli

The systems our tickets are about

Tracklane	Bookings, tracking, customs documents and the driver app. If it’s slow, trucks wait.
Ledgerline	General ledger and period close. Busy on the last days of every month.
Procura	Purchase orders. People ask us to reopen them more than you’d think.
InvoiceHub	Imports supplier invoices in batches. “Did it finish?” is a daily question.
PayGrid	Payroll. Rare tickets, but every one needs a root cause.
Gatehouse and Harbor MDM	Access for everyone. Employees’ laptops sit in Intune; contractors’ go through Harbor MDM.

© 2026 Devansh Parapalli

By the end, you can say **which problems need AI**, and which don’t.

Every class ends by putting its topic somewhere on a card with seven places. We build the card in class 2.

© 2026 Devansh Parapalli

Section 2 of 4

Four eras

Same ticket, four methods. Watch what changes.

01
The ticket

02
Rules: a person writes every decision, the machine just follows

03
Classical ML: the machine finds the pattern, a person picks what it looks at

04
Deep learning: the machine also works out what to look at

© 2026 Devansh Parapalli

The ticket

InvoiceHub import from Armadillo Security Services stuck. Batch IH-260301-07 shows ERR-IH-0413 since 02:00. AP can't start matching.

TKT-204117

© 2026 Devansh Parapalli

Rules: a person writes every decision, the machine just follows

Somebody writes it out. If the text says “InvoiceHub” or “invoice import”, send it to the invoice queue. If it mentions a PO number and “reopen”, send it to procurement.

Nothing to train, nothing to guess. Every decision reads back line by line.

Works well when the logic is known and stable, and an auditor has to follow it.

Falls over when someone writes “armadillo stuff still not there??” and no rule saw that coming.

© 2026 Devansh Parapalli

Thresholds, alert routing and compliance checks should stay rules. Being auditable beats being clever there.

Classical ML: the machine finds the pattern, a person picks what it looks at

Feed it two years of service desk tickets with the queue that fixed each one. It works out which words point to which queue.

You still choose the inputs: the words, the system named, whether it’s period end.

Works well when there’s labelled history and the input is a table or short text. Cheap, fast, runs on a laptop.

Falls over when there are no labels, or a new queue shows up and everything needs relabelling.

© 2026 Devansh Parapalli

Deep learning: the machine also works out what to look at

Stack enough layers and the early ones find simple patterns, the later ones combine them. Letters, then words, then phrases, then meaning.

Nobody hand-picks the inputs any more. Scanned background-check invoices from A.S.S. can go straight in.

Works well when the input is raw text, images or audio and there's a lot of it.

Falls over when labels run short. It needs far more of them, needs GPUs to train, and is hard to explain.

© 2026 Devansh Parapalli

Generative AI: nobody trains for your task, you describe it

The model already learned language by predicting the next word across a huge amount of text. You write the task in plain words: route this ticket to the right queue.

Same model can summarise it, draft the reply to AP, find the runbook.

Works well when the input varies and nobody has labels.

Falls over when it's confidently wrong, gives a different answer next time, costs money per call, or sends data somewhere it shouldn't go (class 8).

© 2026 Devansh Parapalli

Same ticket. **Only the method changed.**

Rules, then learned patterns, then learned inputs, then described tasks. Each era is cheaper to start and harder to pin down than the one before.

© 2026 Devansh Parapalli

Section 3 of 4

Training and inference

Two very different jobs that people call by the same name.

01

Training builds the model once; inference uses it every time someone asks

02

Four things generative AI is not

03

Calling a model is an HTTP request with text in and text out

© 2026 Devansh Parapalli

Training builds the model once; inference uses it every time someone asks

Training

- Done by a model provider, rarely by us
- Weeks, huge amounts of text, thousands of GPUs
- Produces a file of numbers: the weights

Inference

- Done every time we send a request
- Milliseconds to seconds, one prompt at a time
- Costs per token, and that bill is ours

© 2026 Devansh Parapalli

Four things generative AI is not

A database	It doesn't look facts up. It predicts likely text, which is often right and sometimes invented.
Deterministic	Ask twice, get two answers. Settings reduce this; they don't remove it.
A search engine	It knows what it was trained on, up to a date. LOL's runbooks have to be handed to it (class 9).
Magic	It's a very large pattern predictor. Useful, and wrong in patterned ways we can learn to spot.

© 2026 Devansh Parapalli

Calling a model is an HTTP request with text in and text out

1

Get an API key and keep it out of the code

2

Send a message: who you are, what you want

3

The provider runs inference on its machines

4

Text comes back, with a count of tokens used

© 2026 Devansh Parapalli

Section 4 of 4

Code walkthrough

For developers and support. Everyone else: the recording stays up.

01

The first call is a few lines, and most of them are setup

02

Streaming shows the answer arriving token by token

03

Ask the same thing twice and the two answers differ

04

Where today sits on the card

© 2026 Devansh Parapalli

The first call is a few lines, and most of them are setup

code/class01/first_call.py

```
TICKET = "TKT-204117: InvoiceHub import for Armadillo Security Services shows ERR-IH-0413 since 02:00."

reply = client().chat.completions.create(
    model=os.environ["LLM_MODEL"],
    messages=[{"role": "user", "content": f"Which support queue should handle this ticket? {TICKET}"}],
)
print(reply.choices[0].message.content)
```


© 2026 Devansh Parapalli

Streaming shows the answer arriving token by token

code/class01/stream.py

```
stream = client().chat.completions.create(model=os.environ["LLM_MODEL"], messages=messages, stream=True)

for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="", flush=True)
```


© 2026 Devansh Parapalli

Ask the same thing twice and the two answers differ

Run 1
“This is an InvoiceHub import status issue. Send it to the invoice-import queue and check batch IH-260301-07.”

Run 2
“Invoice import. ERR-IH-0413 usually means the batch file failed validation, so AP can’t match yet.”

Both are fine here. In class 7 we make the output something a program can check, so “fine” stops being a judgement call.

© 2026 Devansh Parapalli

Where today sits on the card

1. A script	**The steps are known and don’t change.**
2. A rule	**The decision is known and must be explainable line by line.**
3. Classical ML	**Labelled history, and the input is a table or short text.**
4. A small trained model	**Lots of text, images or audio, enough labels, cost per call matters.**
5. An LLM with a prompt	**Varied language in and out, few labels, someone checks the output.**
6. An LLM plus retrieval, tools or agents	Needs private knowledge, has to act, or takes several steps.
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Run `code/class01/first_call.py` against the course key.
2. Read the class 2 pre-reading: one page on writing a problem down.
3. Bring one problem from your own work, written as a single sentence.

Questions after class: the course channel.

© 2026 Devansh Parapalli